Switchboard

An OpenAI-compatible LLM gateway with routing, fallback, caching, and cost tracking

Shreyas Nimbalkar
October 2026
Version 0.1

Contents

Part 1: The Project in Simple Words 3
- What is this? . . . 3
- The problem it solves . . . 3
- The hospital analogy . . . 4
- How a request travels, step by step . . . 4
- What happens when things go wrong . . . 5
- Streaming . . . 6
- About "estimated cost" . . . 6
- Key ideas, in plain words . . . 7

Part 2: How It Is Built 8
- Big picture . . . 8
- Components . . . 8
- Request lifecycle (non-streaming) . . . 8
- Request lifecycle (streaming) . . . 9
- Error handling . . . 9
- Design decisions . . . 10
- What gets logged . . . 11
- Testing strategy . . . 11

Part 3: Setup and Usage 13
- What you need . . . 13
- Step 1: Install the Python environment . . . 13
- Step 2: Install Ollama and download a model . . . 13
- Step 3: Get free API keys . . . 13
- Step 4: Put the keys in .env . . . 14
- Step 5: Check every provider . . . 14
- Step 6: Run the gateway . . . 14
- Using the gateway from code . . . 14
- API reference . . . 15
- Configuration (config.yaml) . . . 16
- Troubleshooting . . . 16

Part 4: What Was Built, and Why 17
- Build log . . . 17
- Roadmap . . . 19
- Known limitations . . . 20

Part 1: The Project in Simple Words

What is this?

Switchboard is an **LLM gateway**: a **smart receptionist for AI models**.

An app sends a question to one address. The gateway decides **which AI model should answer it**, keeps working when a model fails, remembers answers it has already given, and writes down what every answer cost and how long it took.

Apps never talk to the AI providers directly. They only talk to the gateway.

```
your app  --->  Switchboard  --+-->  Groq        (fast, cheap)
                               +-->  OpenRouter  (smarter)
                               +-->  Gemini      (backup)
                               +-->  Ollama      (runs on your own laptop)
```

The problem it solves

Imagine a company whose AI chatbot answers **1 million questions a day**.

- Most questions are easy: *"What are your store hours?"*
- A few are hard: *"Debug this code"* or *"Compare these two contracts."*

Many companies send **every** question to their most expensive, smartest model. That is like hiring a **surgeon to put on band-aids**. It works, but it wastes money.

They also hit four other problems:

Problem	What goes wrong
A provider goes down	The whole app breaks
Too many requests ("rate limited")	The provider blocks you and the app breaks
The same question asked again and again	You pay for the same answer every time
No measurement	Nobody knows what each feature costs or how slow it is

The gateway fixes all five.

A worked example (hypothetical numbers)

Using the reference prices in this project's `config.yaml`:

- "Smart" model (Nemotron 3 Ultra): $0.50 per million input tokens, $2.20 per million output tokens
- "Fast" model (gpt-oss-20b): $0.018 per million input tokens, $0.09 per million output tokens

Say each question uses 500 input tokens and 200 output tokens, and there are 1,000,000 questions a day.

Strategy	Daily cost
Send everything to the smart model	about $690
Send everything to the fast model	about $27 (but hard questions get worse answers)

Strategy	Daily cost
Route 80% to fast, 20% to smart	about $160

Routing saves about **77%** in this example, while hard questions still get the smart model. The real savings depend on your traffic. Measuring them properly is what the evaluation step on the roadmap is for.

The hospital analogy

At a hospital	In the gateway
A patient walks in	An app sends a question
The front desk checks how serious it is	The **router** decides: easy or hard?
A cold goes to a nurse, a heart problem to a specialist	Easy goes to the **fast** tier, hard to the **smart** tier
The specialist is busy, so the patient sees another one	**Fallback**: if a provider fails, try the next one
"We treated this exact case an hour ago"	**Cache**: same question, reuse the saved answer
Every visit goes in a logbook	**Logging**: every request saved with cost and time
A monthly report to management	The **/stats** page

How a request travels, step by step

Example: an app sends **"Write code to reverse a list"** and asks for model `auto`.

Step 1. The request arrives. The app sends the question to `/v1/chat/completions`. This is the **same format as OpenAI's API**, so any app built for the OpenAI API can use the gateway by changing one URL.

Step 2. Password check (optional). If a gateway password is set, requests without it are rejected. This matters once the gateway is on the internet.

Step 3. The router picks a tier. The app asked for `auto`, so the router reads the question:

- Contains words like *code*, *debug*, *step by step*, *analyze*? Use **smart**.
- Longer than 4,000 characters? Use **smart**.
- A long conversation (more than 6 user messages)? Use **smart**.
- Otherwise, use **fast**.

"Write **code** to reverse a list" contains "code", so it goes to **smart**. (Apps can skip this by asking for `fast`, `smart`, or an exact model name.)

Step 4. Check the cache. Has this *exact* question been asked recently? If yes, return the saved answer instantly, for free. If no, continue.

Step 5. Call the AI, with backup plans. "smart" is an ordered **list of backups**:

```
smart:  1. OpenRouter  (Nemotron 3 Ultra 550B)
        2. Groq        (gpt-oss-120b)
        3. Gemini      (3.5 Flash)
        4. Ollama      (Llama 3.2 3B, on your laptop)
```

The gateway tries number 1. What happens next depends on the result:

Result	What the gateway does
It works	Done
Timeout or server error (a temporary glitch)	Wait 0.5s and retry, then 1s and retry, then move on
"Too many requests" (429)	Put that provider in a 30-second time-out, move to the next one now
Cannot connect at all	Move to the next one immediately (retrying will not help)
No API key set for it	Skip it

Step 6. Send the answer back, with labels. The answer goes back with extra **headers** that say what happened:

```
X-Gateway-Route:        rule:keyword:code   <- why it was routed this way
X-Gateway-Alias:        smart               <- which tier
X-Gateway-Provider:     openrouter          <- who actually answered
X-Gateway-Attempts:     1                   <- how many tries it took
X-Gateway-Cache:        miss                <- served from cache?
X-Gateway-Est-Cost-USD: 0.00004120          <- estimated cost
```

Step 7. Write it in the logbook. Every request is saved to a small database file (`gateway.db`): which model answered, tokens used, estimated cost, total time, time until the first word, and any errors.

Step 8. Reports. `/stats` shows totals and breakdowns: requests, errors, cache hits, typical and slow response times (p50 / p95), and cost, per provider, per tier, and per routing rule.

What happens when things go wrong

Scenario A: Groq is having an outage. A "fast" request goes to Groq. Groq returns server errors three times in a row. The gateway gives up on Groq and sends the same request to the next backup, Gemini. The user still gets an answer; the header says `X-Gateway-Provider: gemini` and `X-Gateway-Attempts: 4`.

Scenario B: you hit Groq's free-tier limit. Groq replies "429 Too Many Requests, retry after 16 seconds". The gateway does *not* retry right away (that would make it worse). It puts **that one model** in a 16-second time-out and moves on to the next option. Groq's other models keep working, because Groq counts each model's limit separately.

If *every* option is in a time-out, the gateway does not give up. It **waits in a queue** until the first one frees up (up to 90 seconds), then tries again. The header `X-Gateway-Queued-Ms` says how long the request waited.

Scenario C: no keys at all. Every cloud provider is skipped because it has no key, and Ollama answers everything. This is how the project ran on day one, before any keys were added.

Scenario D: everything is down. The gateway returns an error that lists every attempt and why it failed, for example:

```
groq/openai/gpt-oss-20b: skipped (no GROQ_API_KEY in .env)
ollama/llama3.2:3b: unreachable - cannot connect
```

Streaming

Streaming means the answer appears **word by word**, like in ChatGPT, instead of all at once at the end. The gateway supports it.

The tricky part: once the first word has been sent to the user, you cannot switch to a different provider mid-sentence. So the gateway makes sure a provider has **accepted** the request before it sends anything to the user. That way, streaming requests get the same backup plans as normal requests.

About "estimated cost"

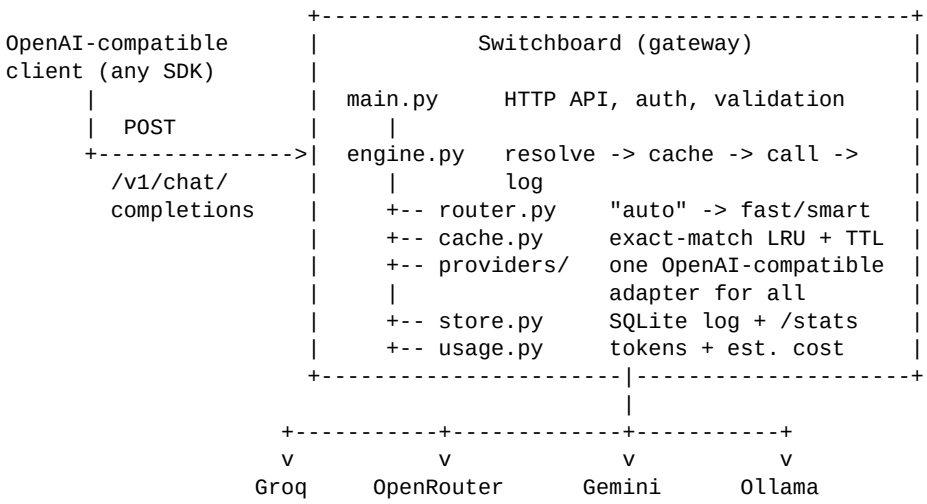
On free tiers the real bill is $0. To still measure savings, the gateway calculates what each request **would cost** at the model's normal paid price (the `prices` section in `config.yaml`). That is the same way companies estimate cost when they plan.

Key ideas, in plain words

Term	Meaning
LLM	Large language model: the AI that reads and writes text
API	A way for one program to talk to another over the internet
API key	A password that identifies you to an AI provider
Token	A small piece of text, roughly 3/4 of a word. Providers charge per token
Provider	A company that hosts AI models (Groq, OpenRouter, Google, ...)
Alias / tier	A nickname like "fast" or "smart" that maps to a list of real models
Deployment	One specific model on one specific provider, e.g. `groq/openai/gpt-oss-20b`
Router	The part that decides which tier handles a request
Fallback	Trying the next option when one fails
Retry with backoff	Trying again after waiting longer each time (0.5s, 1s, 2s, ...)
Rate limit (429)	The provider saying "slow down, too many requests"
Cooldown	Temporarily skipping a model that rate-limited you
Retry-After	A header where the provider says exactly how long to wait
Wait queue	Holding a request until a rate-limited model frees up, instead of failing
Cache	Saved answers that can be reused
Latency	How long a response takes
p50 / p95	Half of requests are faster than p50; 95% are faster than p95
TTFT	Time to first token: how long until the first word appears
Streaming	Sending the answer word by word as it is generated
SSE	Server-Sent Events: the format used to stream the words
Ollama	Free software that runs AI models on your own computer
Eval set	A list of test questions with known good answers, used to score quality

Part 2: How It Is Built

Big picture



Components

File	Responsibility
`gateway/main.py`	FastAPI app: endpoints, request validation, optional API-key auth
`gateway/engine.py`	The core: model resolution, retry/fallback/cooldown, caching, logging, streaming relay
`gateway/router.py`	Rules-based router for model `auto`
`gateway/cache.py`	In-memory exact-match cache (LRU eviction + TTL expiry)
`gateway/providers/openai_compat.py`	HTTP adapter for any OpenAI-compatible provider; classifies errors
`gateway/store.py`	SQLite request log and the `/stats` aggregation (p50/p95, cost, errors)
`gateway/usage.py`	Token estimation and cost-at-list-price calculation
`gateway/config.py`	Loads `config.yaml` and `.env` into typed settings
`config.yaml`	Providers, aliases (fallback chains), prices, routing rules, reliability knobs

Request lifecycle (non-streaming)

1. **Validate.** `main.py` parses the body with Pydantic. `messages` must be non-empty. Unknown OpenAI fields (`top_p`, `tools`, ...) are kept and passed through.
2. **Resolve.** `engine.resolve()` turns the requested model into a fallback chain:
   - `auto`: the router picks an alias and records why (e.g. `rule:keyword:code`)
   - an alias (`fast`, `smart`, `local`): its chain from `config.yaml`

   - `provider/model`: a one-item chain, sent directly
   - anything else: HTTP 404 with the list of valid options
3. **Cache lookup.** The key is a SHA-256 hash of the whole request (minus `stream`). A hit returns immediately and is logged with `cache_hit=1` and cost 0.
4. **Call with fallback.** `_call_with_fallback()` walks the chain:
   - skip deployments whose provider has no API key
   - skip models that are cooling down after a 429
   - if every model is rate-limited, wait for the first cooldown to end (up to `max_queue_wait_s`) and try the chain again
   - call the provider; on failure, act on the error kind (table below)
5. **Account.** Use the provider's reported token usage, or estimate it (about 4 characters per token) if missing. Compute estimated cost.
6. **Log** one row to SQLite, **store** the response in the cache, and **return** it with `X-Gateway-*` headers.

Request lifecycle (streaming)

Streaming splits into two phases, because fallback is only possible before the first byte reaches the client:

1. `open_stream()` runs the same fallback loop, but each attempt only *opens* the stream and checks the HTTP status. A failing provider is skipped exactly like in the non-streaming path.
2. `relay()` passes Server-Sent Events through unchanged while measuring time to first token and collecting the output. The gateway asks providers for usage in the final chunk (`stream_options.include_usage`); if a provider does not send it, tokens are estimated. If the provider dies mid-stream, the client receives an error event and the request is logged as an error.

Error handling

The provider adapter turns every failure into a `ProviderError` with a **kind**. The engine reacts to the kind:

Kind	Triggered by	Gateway action
`transient`	timeout, network blip, HTTP 5xx	retry same deployment with backoff (0.5s, 1s), then fall back
`unreachable`	connection refused (e.g. Ollama not running)	fall back immediately, no retry
`rate_limited`	HTTP 429	cool **that model** down for the `Retry-After` time (30s default), fall back immediately
`auth`	HTTP 401 / 403	fall back (key wrong or revoked)
`bad_request`	HTTP 400 / 404 / 422	fall back, no retry (e.g. model ID retired)
`unconfigured`	no API key in `.env`	skip without calling

If every deployment fails, the gateway picks the most useful status code:

Situation	Status
Nothing callable (no keys, local model down)	503 Service Unavailable
Every real attempt was a bad request	400 Bad Request
Every real attempt was rate limited	429 Too Many Requests
Mixed failures	502 Bad Gateway

The error body lists every attempt with its reason, so failures are easy to debug.

Design decisions

One adapter for every provider. Groq, OpenRouter, Gemini and Ollama all expose an OpenAI-compatible `/chat/completions` endpoint. One adapter keeps the code small; adding a provider is a config change, not new code.

Fallback chains live in config, not code. Changing the order of providers or adding a backup is a YAML edit, so the behavior can be tuned without redeploying code.

Retry only what retrying can fix. Timeouts and 5xx errors are often temporary, so they are retried. A 429 means "slow down"; retrying immediately makes it worse, so the model is cooled down instead. A refused connection will not fix itself in half a second, so it falls back at once.

Cooldowns are per model, not per provider. Providers such as Groq meter each model separately (on the free tier: 1,000 requests a day and 8,000 tokens a minute *per model*). Cooling down the whole provider when one model hits its limit would wrongly block models that still have capacity.

Trust the provider's Retry-After. Per-minute token limits usually clear in seconds, and the 429 response says exactly when. Using that value instead of a fixed 30 seconds brings the model back as soon as it is usable.

Queue instead of failing when everything is busy. Long-running clients (agents, benchmark runs) would rather wait 20 seconds than lose a half-finished conversation. When every option is rate-limited, the gateway sleeps until the earliest cooldown ends and tries again, up to a configurable budget (`max_queue_wait_s`, 90s in `config.yaml`; 0 disables it). It never waits for a cooldown that would end after the budget, and it never queues for other errors.

Exponential backoff. Waiting longer between each retry (0.5s, 1s, 2s) gives a struggling provider room to recover instead of piling more load on it.

Streaming fallback before the first byte. Once tokens reach the client, switching providers would produce a broken, mixed answer. Checking the status before relaying anything keeps streaming requests reliable.

Cache key = the whole request. Different `temperature`, `max_tokens` or system prompts can change the answer, so they are part of the key. Only `stream` is ignored, because it changes the delivery, not the content. Clients can opt out per request with `X-Gateway-Cache: bypass`.

SQLite for the request log. Zero setup, a single file, and SQL for analysis. It fits a single-instance gateway; a multi-instance deployment would move to Postgres or a metrics system.

Estimated cost at list price. Free tiers cost $0, which hides the real tradeoff. Pricing each call at the model's paid list price makes routing savings measurable and comparable.

Rules router first, as a baseline. Keyword rules are simple and explainable. They are the baseline a learned classifier must beat on the eval set (roadmap step 5).

Built for testing. The HTTP transport, clock and sleep function are injected. Tests plug in a fake upstream and skip real waiting, so the whole failure-handling suite runs in under a second with no network.

What gets logged

One row per request in the `requests` table of `gateway.db`:

Column	Meaning
`ts`	Unix timestamp
`requested_model`	What the client asked for (`auto`, `fast`, ...)
`alias`, `routed_by`	Which tier was chosen and why
`provider`, `model`	Who actually answered
`stream`, `cache_hit`	Streaming request? Served from cache?
`status`, `attempts`, `error`	Outcome, number of tries, failure details
`queued_ms`	Time spent waiting in the rate-limit queue
`latency_ms`, `ttft_ms`	Total time and time to first token
`prompt_tokens`, `completion_tokens`	Token usage
`usage_estimated`	1 if tokens were estimated rather than reported
`est_cost_usd`	Estimated cost at list price

Testing strategy

The test suite (`tests/`) uses a **fake upstream provider** that can be scripted to fail on command, for example "return 500 three times, then succeed". Each test then checks what the gateway did: how many calls each provider received, which provider served the answer, and what was logged.

Covered behaviors:

- alias, `auto`, and direct `provider/model` resolution; unknown models
- pass-through of extra OpenAI parameters; input validation
- retries on transient errors; fallback after retries run out
- no retry on refused connections; per-model cooldown after 429; Retry-After parsing (seconds and HTTP-date forms); the wait queue (succeeds, gives up past its budget, ignores non-rate-limit errors, works for streaming); queued time in the log; and migration of log files created before the `queued_ms` column existed
- skipping providers without keys; final status codes (400 / 429 / 502 / 503)

- cache hits, cache bypass, and parameter-sensitive cache keys
- streaming pass-through, usage capture, usage estimation, streaming fallback
- `/stats` aggregation, `/v1/models`, and API-key auth
- unit tests for the router, cache (TTL + LRU), cost math, and percentiles

Run them with `python -m pytest -q`. They also run automatically on every push through GitHub Actions (`.github/workflows/tests.yml`).

Part 3: Setup and Usage

What you need

Item	Why	Where	Cost
Python 3.11+	Runs the gateway	https://www.python.org/downloads	Free
Ollama	Runs a small model locally: the free, unlimited fallback	https://ollama.com/download	Free
Groq API key	Fast cloud models (the "fast" tier)	https://console.groq.com	Free tier, no card
OpenRouter API key	Many free models behind one key (the "smart" tier)	https://openrouter.ai	Free models, no card
Gemini API key (optional)	Extra backup provider	https://aistudio.google.com	Free tier

Free tiers have strict rate limits and may use your prompts for training. Only send test or synthetic data through them.

Step 1: Install the Python environment

```
git clone https://github.com/shreyasnimbalkar/switchboard-llm-gateway.git
cd switchboard-llm-gateway
python -m venv .venv
.venv\Scripts\python -m pip install -r requirements-dev.txt
```

(On macOS/Linux use `.venv/bin/python` instead of `.venv\Scripts\python`.)

Step 2: Install Ollama and download a model

1. Download and install Ollama from https://ollama.com/download.
2. Download the model (about 2 GB):

```
ollama pull llama3.2:3b
```

The first request after Ollama starts can take a minute or more while the model loads into memory. Later requests take a few seconds.

Step 3: Get free API keys

Groq: sign in at https://console.groq.com, open **API Keys**, click **Create API Key**, and copy it (starts with `gsk_`). It is only shown once.

OpenRouter: sign in at https://openrouter.ai, open your profile menu, then **Keys**, then **Create Key** (starts with `sk-or-`).

Gemini (optional): sign in at https://aistudio.google.com and click **Get API key**, then **Create API key**.

Step 4: Put the keys in `.env`

```
copy .env.example .env
notepad .env
```

Paste each key after its `=` sign, with no quotes or spaces:

```
GROQ_API_KEY=gsk_...
OPENROUTER_API_KEY=sk-or-...
GEMINI_API_KEY=
GATEWAY_API_KEY=
```

`.env` is listed in `.gitignore`, so it is never uploaded to GitHub.

Step 5: Check every provider

```
.venv\Scripts\python scripts\check_providers.py
```

This sends one tiny prompt to every model in `config.yaml` and prints a table of which ones work and, if not, why (missing key, wrong key, outdated model ID, not running).

Step 6: Run the gateway

```
.venv\Scripts\uvicorn gateway.main:create_app --factory --reload
```

Then, in a second terminal, send some sample prompts:

```
.venv\Scripts\python scripts\demo.py
```

Useful pages while it runs:

- Interactive API docs: http://127.0.0.1:8000/docs
- Statistics: http://127.0.0.1:8000/stats
- Provider health: http://127.0.0.1:8000/health

Using the gateway from code

Any OpenAI client works. Point it at the gateway:

```
from openai import OpenAI

client = OpenAI(base_url="http://127.0.0.1:8000/v1", api_key="unused-locally")
reply = client.chat.completions.create(
    model="auto",   # or "fast", "smart", "local", or "groq/openai/gpt-oss-20b"
    messages=[{"role": "user", "content": "Explain recursion in one sentence."}],
)
print(reply.choices[0].message.content)
```

Or with plain HTTP:

```
curl http://127.0.0.1:8000/v1/chat/completions -H "Content-Type: application/json" \
  -d "{\"model\": \"auto\", \"messages\": [{\"role\": \"user\", \"content\": \"hi\"}]}"
```

API reference

Method	Path	Purpose
POST	`/v1/chat/completions`	Chat completion (OpenAI format), streaming supported
GET	`/v1/models`	Lists `auto`, the aliases, and every deployment
GET	`/health`	Which providers are configured; current cooldowns; cache size
GET	`/stats`	Requests, errors, cache hits, p50/p95 latency, tokens, est. cost
GET	`/logs/recent?limit=50`	The most recent logged requests

`/health` is always public. If `GATEWAY_API_KEY` is set, every other endpoint requires the header `Authorization: Bearer <GATEWAY_API_KEY>`.

Model names you can request

Model	Meaning
`auto`	The router chooses `fast` or `smart`
`fast`	Cheap, quick models first
`smart`	Stronger models first
`local`	Only the local Ollama model
`provider/model`	Exactly that model on that provider, no fallback

Response headers

Header	Meaning
`X-Gateway-Route`	Why this tier was chosen (`rule:default`, `rule:keyword:code`, `explicit`, `direct`)
`X-Gateway-Alias`	The tier used
`X-Gateway-Provider` / `X-Gateway-Model`	Who actually answered
`X-Gateway-Attempts`	Number of provider calls it took
`X-Gateway-Queued-Ms`	Time spent waiting for a rate limit to clear (only when it waited)
`X-Gateway-Cache`	`hit` or `miss`
`X-Gateway-Est-Cost-USD`	Estimated cost at list price (non-streaming)

Send `X-Gateway-Cache: bypass` to skip the cache for a request.

Configuration (`config.yaml`)

Section	What it controls
`providers`	Base URL, which env var holds the key, timeout, extra headers
`aliases`	Each tier's fallback chain, tried in order
`prices`	List price per million tokens, used for cost estimates
`routing`	Rules for `auto`: long-prompt length, conversation length, keywords
`reliability`	Retries, backoff base, default rate-limit cooldown, wait-queue budget
`cache`	Time-to-live and maximum entries
`db_path`	Where the SQLite request log is stored

Provider model IDs change over time. If `check_providers.py` reports `bad_request` for a model, find its current ID in the provider's dashboard and update `config.yaml`.

Troubleshooting

Symptom	Likely cause	Fix
`skipped (no GROQ_API_KEY in .env)`	Key missing	Add it to `.env`; restart the gateway
`unreachable ... cannot connect` for Ollama	Ollama not running	Start the Ollama app
`model 'llama3.2:3b' not found`	Model not downloaded	`ollama pull llama3.2:3b`
`bad_request (404)` for a cloud model	Model ID retired or renamed	Update the ID in `config.yaml`
`auth (401)`	Key wrong or revoked	Create a new key
`rate_limited (429)`	Free-tier limit reached	Wait; the gateway falls back meanwhile
First request takes over a minute	Ollama loading the model	Normal; later requests are fast
`'ollama' is not recognized` in a terminal	Terminal opened before install	Open a new terminal

Part 4: What Was Built, and Why

Build log

Step 1: Core proxy

What: An OpenAI-compatible `/v1/chat/completions` endpoint that forwards requests to Groq, OpenRouter, Gemini or Ollama, with streaming support.

Why: Being OpenAI-compatible means any existing app or SDK can use the gateway by changing one URL. Without that, nobody would adopt it.

Step 2: Request logging

What: Every request is recorded in SQLite with tokens, estimated cost, total latency and time to first token. `/stats` summarizes it per provider, tier and routing rule.

Why: You cannot claim savings or speed you have not measured. This log is the data source for the evaluation and dashboard in steps 5 and 6.

Step 3: Reliability

What: Retries with exponential backoff for temporary errors, fallback chains across providers, a cooldown for rate-limited models, and fallback for streaming requests before the first byte.

Why: Free tiers rate-limit often and providers have outages. The gateway's job is to keep apps working when that happens.

Step 3b: Rate-limit upgrade

What: Three changes, made while preparing Switchboard to carry long agent benchmark runs on free tiers:

1. **Per-model cooldowns.** The first version cooled down a whole *provider* after a
   429. Measuring Groq's free tier showed its limits are *per model* (1,000 requests a day and 8,000 tokens a minute each), so one busy model was wrongly blocking the others. Cooldowns are now tracked per `provider/model`.
2. **Retry-After.** The cooldown now uses the provider's own `Retry-After` header (seconds or HTTP-date) instead of a fixed 30 seconds.
3. **Wait queue.** If every option is rate-limited, the request waits for the first one to free up (up to `max_queue_wait_s`, 90s) instead of failing. Waiting time is returned in `X-Gateway-Queued-Ms` and logged in a new `queued_ms` column; old log files are migrated automatically.

Why: An agent conversation that fails halfway because of a 20-second rate limit is wasted work. Waiting is the better trade for long-running clients.

Verified: 13 new tests (52 total). Against the real Groq API, 6 parallel ~2,400-token requests were sent to one model, more than its 8,000 tokens/minute limit: 5 went straight through (about 0.7 to 1 second each) and 1 was rate-limited. Groq asked for a wait of about 16 seconds, the gateway queued it, and it then succeeded. Before the change, that request would have failed.

Extras added along the way

Feature	Why
`auto` routing with keyword rules	The baseline for comparing smarter routers later
Exact-match cache	Repeated questions become instant and free
`X-Gateway-*` response headers	Shows exactly what happened for each request
Optional gateway API key	Required before putting the gateway on the internet
`scripts/check_providers.py`	One command to verify every key and model ID
`scripts/demo.py`	A quick live demonstration of routing, caching and cost
Offline test suite + GitHub Actions	Proves the failure handling works, on every push

Verification so far

- The automated test suite passes (52 tests at the time of writing).
- The gateway was started for real with no keys and Ollama not running. It returned a clear error naming each missing piece.
- With Ollama running `llama3.2:3b` locally and no cloud keys, the demo sent 5 prompts:

Prompt	Routed to	Reason	Outcome
"What is the capital of Japan?"	fast	`rule:default`	Answered ("Tokyo")
"Write code for a ... palindrome"	smart	`rule:keyword:code`	Answered with working Python
"Give me a synonym for 'happy'."	fast	`rule:default`	Answered ("joyful")
"Explain why the sky is blue, step by step."	smart	`rule:keyword:step by step`	Answered
"What is the capital of Japan?" (again)	fast	`rule:default`	**Cache hit**

Result: 0 errors, p95 latency 3.8 seconds on a laptop GPU (GTX 1650). With no cloud keys, both tiers fell back to the local model, which is the fallback design working as intended.

Connecting real providers

When the free API keys were added, `check_providers.py` showed that every key worked but **every cloud model ID in the original config had been retired or moved off the free tier**. The fix was to query each provider's live model list, test candidates, and pick replacements on speed and list price:

Tier	Chosen model	Why
fast	Groq `openai/gpt-oss-20b`	Under 0.5s, and the cheapest list price tested
fast backup	Gemini `gemini-3.1-flash-lite`	Fast (under 1s), different provider
smart	OpenRouter `nvidia/nemotron-3-ultra-550b-a55b:free`	Largest free model available

Tier	Chosen model	Why
smart backups	Groq `openai/gpt-oss-120b`, Gemini `gemini-3.5-flash`	Strong models on other providers

A model that answered faster (`qwen3.8-27b`) was rejected for the fast tier because its list price was about 10x higher per request, which would defeat the purpose of a cheap tier. Lesson: provider model catalogs change quickly, so model IDs belong in config and need a health check.

The same demo through the real providers:

Prompt	Served by	Est. cost
"What is the capital of Japan?"	Groq gpt-oss-20b	$0.000005
"Write code for a ... palindrome"	OpenRouter Nemotron 3 Ultra	$0.001336
"Give me a synonym for 'happy'."	Groq gpt-oss-20b	$0.000005
"Explain why the sky is blue, step by step."	OpenRouter Nemotron 3 Ultra	$0.001334
"What is the capital of Japan?" (again)	Cache	$0

Observations:

- The fast tier answered with a median latency of about 0.5 seconds; streaming worked, with the first token after about 0.66 seconds.
- A smart-tier request cost about **270x** more than a fast-tier one at list prices, which is why routing easy prompts away from it matters.
- The smart tier took 17 to 29 seconds for long answers on the free tier. Free-tier capacity is slow, and long answers take longer. This is a real tradeoff to measure in the evaluation step.

Roadmap

Step 4: Semantic cache

What: Reuse answers for questions that *mean* the same thing, not just identical text. "What's the capital of Japan?" and "Japan's capital city?" should match.

How: Turn each question into an **embedding** (a list of numbers that captures its meaning). For a new question, find the most similar cached one using cosine similarity. If the similarity is above a threshold, reuse its answer. Move the cache to Redis so it survives restarts.

Why: More cache hits mean lower cost and latency. The threshold is a real tradeoff: too loose and users get wrong answers, too strict and nothing matches. Measuring that tradeoff is part of the step.

Step 5: Learned router

What: Replace keyword rules with a small classifier that predicts whether a prompt needs the smart tier.

How: Label a set of prompts by whether the fast model's answer was good enough, then train a lightweight classifier on embeddings of those prompts.

Why: Keywords are crude. "Write a haiku about code" is easy but contains "code". A learned router should send fewer easy prompts to the expensive tier.

Step 6: Evaluation and dashboard

What: A set of about 200 prompts with reference answers, scored automatically, and a dashboard of quality versus cost versus p95 latency for each routing strategy: always-smart, always-fast, rules, and learned.

Why: This produces the headline result: *"routing cut estimated cost by X% while keeping Y% of quality."*

Step 7: Deploy and write up

What: Host the gateway publicly (with `GATEWAY_API_KEY` set), add the results and charts to the README, and publish a short write-up.

Why: A live demo and real numbers are what make the project credible.

Known limitations

Limitation	Impact	Planned fix
Cache is in memory	Lost on restart; not shared between instances	Redis (step 4)
Keyword router is crude	Some easy prompts go to the smart tier, and vice versa	Learned router (step 5)
Token counts are sometimes estimated	When a provider omits usage, cost is approximate	Flagged with `usage_estimated` in the log
Costs are estimates at list price	Not a real bill	Prices are configurable in `config.yaml`
SQLite log	Fine for one instance, not for a fleet	Postgres or a metrics system if scaled
Cooldown state is per process	Multiple instances would not share it	Shared store (Redis) if scaled
Free-tier limits	Large eval runs must be spread out	Fallback chains spread the load